STM32F103

Multi-Alarm RTC System

Technical Design Document

Version	2.0
Date	2026-03-24
Target MCU	STM32F103C8T6 (Blue Pill)
Toolchain	Keil MDK / ARM-GCC + STM32 Standard Peripheral Library
Clock Source	LSE 32.768 kHz Crystal
Status	Released

1. Project Overview 2
1.1 Key Features 2
1.2 Hardware Summary 3
2. Module Architecture 3
2.1 Dependency Graph 3
3. Alarm Configuration 4

3.1 AlarmSlot Structure....4
3.2 Constraints and Behaviour....4
3.3 Scheduling Algorithm....4
4. Finite State Machine....4
4.1 States....5
4.2 Events....5
4.3 State Transition Table....5
4.4 Main Loop Structure....6
5. RTC and Timekeeping....6
5.1 Clock Source and Initialisation....6
5.2 Time Zone Strategy....6
5.3 Known Limitation: Year 2038....7
6. USART Protocol....7
6.1 Frame Format....7
6.2 Receiver State Machine....7
6.3 NTP Sync Payload....8
7. OLED Display Layout....8
7.1 Row 4 State Strings....8
8. Power Management....9
8.1 STANDBY Mode....9
8.2 Wake and Re-initialisation Sequence....9
8.3 IDLE_STANDBY_DELAY_SEC....9
9. Boot Modes....9
10. Known Issues and Limitations....10
11. Revision History....10

1. Project Overview

This document describes the firmware architecture, module design, and operational behaviour of a battery-backed multi-alarm clock built on the STM32F103 microcontroller. The system uses the on-chip Real-Time Clock (RTC) running from an external 32.768 kHz crystal oscillator to maintain timekeeping through power cycles and deep-sleep events. Multiple configurable daily alarm windows drive an LED output and suppress entry into STANDBY mode, maximising battery life between alarms.

1.1 Key Features

- Up to N simultaneous daily alarm slots, each defined by start time and duration
- Cross-midnight alarm windows (e.g. 23:50 for 20 min) fully supported
- Single RTC alarm register dynamically programmed to the nearest upcoming boundary
- Ultra-low-power STANDBY mode between alarm periods (wake via RTC or PA0 WKUP pin)
- NTP-style time synchronisation over USART1 with XOR-checksummed framing
- Explicit finite state machine (FSM) architecture in firmware
- OLED display (SSD1306, I2C) for live status, counter, and alarm information

- Download-mode protection via PB11 pin to prevent accidental standby lockout

1.2 Hardware Summary

OLED SCL	PB8	Output (OD)	I2C clock – SSD1306 OLED
OLED SDA	PB9	Output (OD)	I2C data – SSD1306 OLED
USART1 TX	PB6	Output (AF)	Serial transmit (remapped)
USART1 RX	PB7	Input	Serial receive (remapped)
KEY1 / NTP	PB1	Input (IPU)	High on boot = run NTP sync
KEY2 / DL	PB11	Input (IPU)	High on boot = download mode
LED	PA1	Output (PP)	On during active alarm period
WKUP	PA0	Input	Rising edge wakes STANDBY mode
LSE XTAL	PC14/15	Analog	32.768 kHz RTC clock source

2. Module Architecture

The firmware is organised into six source modules, each with a single responsibility. Dependencies flow strictly downward: higher-level modules call lower-level ones; no module calls back up the stack.

main	main.c	FSM, alarm scheduling, top-level control flow
MyRTC	MyRTC.c/.h	RTC init, time read/write, UTC↔UTC+8 conversion, NTP sync
USART	USART.c/.h	115200 8N1 framed protocol, XOR checksum, ISR state machine
OLED	OLED.c/.h	SSD1306 I2C driver, character/string/number rendering
LED	LED.c/.h	GPIO output abstraction for LED control
KEY	KEY.c/.h	Debounced button input on PB1 and PB11
Delay	Delay.c/.h	SysTick-based us/ms/s blocking delays

2.1 Dependency Graph

```
main.c
  ├── MyRTC.c  ─── USART.c
  ├── OLED.c
```

```
    ├── LED.c
    ├── KEY.c
    └── Delay.c

  (All modules depend on Delay.c and stm32f10x.h)
```

3. Alarm Configuration

Alarms are defined as a static array of AlarmSlot structures at the top of main.c. This is the only place a user needs to edit to add, remove, or modify alarms.

3.1 AlarmSlot Structure

```
typedef struct {
    uint8_t start_hour;        /* 0-23 */
    uint8_t start_minute;      /* 0-59 */
    uint8_t duration_minutes;  /* 1-1439 (cross-midnight: up to 23h59m) */
} AlarmSlot;

static const AlarmSlot ALARMS[] = {
    {  7, 30, 2 },   /* Alarm 1: 07:30 for 2 minutes */
    { 12,  0, 1 },   /* Alarm 2: 12:00 for 1 minute  */
    { 20, 38, 2 },   /* Alarm 3: 20:38 for 2 minutes */
};
```

3.2 Constraints and Behaviour

- Overlapping alarm windows produce undefined behaviour; windows must not overlap.
- ALARM_COUNT is derived automatically via sizeof — no manual count is required.
- Cross-midnight alarms are detected when end_secs > 86400. The active window is split into [start, midnight) and [0, end mod 86400).
- The single RTC_ALR register is always programmed to the nearest upcoming boundary across all slots (either a start or an end), ensuring no alarm is ever missed.

3.3 Scheduling Algorithm

FindNextAlarmBoundary() iterates over every start and end across all AlarmSlot entries and computes the delta in seconds from the current time-of-day to each boundary. The minimum positive delta wins. If all boundaries lie in the past for today, the algorithm automatically wraps to tomorrow by adding 86400 seconds.

4. Finite State Machine

The entire runtime behaviour of main.c is governed by an explicit FSM. State transitions are centralised in FSM_Transition(); ongoing per-tick actions are in FSM_Tick(). This separation

ensures that each state's entry action runs exactly once and that display updates are never duplicated.

4.1 States

STATE_BOOT_CHECK	Reads PB11 and PB1 after reset. Routes to one of three boot outcomes.	Immediate — evaluated once
STATE_BOOT_DOWNLOAD	PB11 high. Infinite loop; SWD remains accessible for flashing.	Never (requires hardware reset)
STATE_BOOT_NTP	PB1 high. Calls NTP_SyncTime() (blocking, 30 s timeout).	NTP completes (success or timeout)
STATE_IDLE	No alarm active. OLED shows live counter. Counts down to standby.	EVT_IN_ALARM_PERIOD or EVT_STANDBY_TIMEOUT
STATE_STANDBY	Calls PWR_EnterSTANDBYMode(). Never returns; wake is a full reset.	N/A — hardware reset on wake
STATE_ALARM	An alarm window is active. LED on. Standby is suppressed.	EVT_IN_ALARM_PERIOD clears (window ends)

4.2 Events

EVT_RTC_ALARM	0x01	RTC_FLAG_ALR	RTC alarm register has fired. Flag is cleared in CollectEvents().
EVT_IN_ALARM_PERIOD	0x02	FindActiveAlarm()	Current time-of-day falls inside at least one AlarmSlot window.
EVT_STANDBY_TIMEOUT	0x04	g_idle_ticks	IDLE tick counter has reached IDLE_STANDBY_DELAY_SEC.

4.3 State Transition Table

BOOT_CHECK	PB11 high	BOOT_DOWNLOAD	Display download message
BOOT_CHECK	PB1 high	BOOT_NTP	Display NTP syncing message
BOOT_CHECK	(neither pin)	IDLE	OnEnter_Idle()
BOOT_NTP	NTP done + EVT_IN_ALARM_PERIOD	ALARM	OnEnter_Alarm()
BOOT_NTP	NTP done + not in alarm	IDLE	OnEnter_Idle()

	period		
IDLE	EVT_IN_ALARM_PERIOD	ALARM	OnEnter_Alarm()
IDLE	EVT_STANDBY_TIMEOUT	STANDBY	OnEnter_Standby()
ALARM	EVT_IN_ALARM_PERIOD clears	IDLE	OnEnter_Idle()
STANDBY	(none — hardware reset on wake)	BOOT_CHECK	Full re-initialisation

4.4 Main Loop Structure

The main while(1) loop executes four fixed steps every second:

```
while (1)
{
    AppEvent evt = CollectEvents();    /* 1. Sample all inputs, clear flags */
    FSM_Transition(evt);               /* 2. Evaluate transitions + entry actions */
    FSM_Tick();                        /* 3. Per-tick actions (display, counters) */
    Delay_ms(1000);                    /* 4. Fixed 1-second tick period */
}
```

5. RTC and Timekeeping

5.1 Clock Source and Initialisation

The RTC is clocked from the LSE (Low-Speed External) oscillator at 32.768 kHz. The prescaler is set to 32767 (0x7FFF), yielding a 1 Hz counter increment. Because the LSE and RTC are in the backup power domain, they continue running in STANDBY mode as long as VBAT or VDD is present.

First-boot detection: BKP_DR1 is checked against the magic value 0xA5A5. If the value is absent (fresh chip or full power loss), the LSE, prescaler, and initial time are programmed and the magic value is written. On all subsequent boots (including wake from STANDBY), only RTC_WaitForSynchro() is called to re-sync the APB shadow registers.

5.2 Time Zone Strategy

- The RTC hardware counter stores UTC time as a Unix timestamp (seconds since 1970-01-01 00:00:00 UTC).
- MyRTC_SetTime() accepts UTC+8 local time, subtracts 8 hours before writing the counter.
- MyRTC_ReadTime() reads the counter, adds 8 hours, and populates MyRTC_Time[].
- MyRTC_CounterToSecsMidnight() adds the UTC+8 offset and returns seconds elapsed since local midnight — used by all alarm calculations.

5.3 Known Limitation: Year 2038

The ARM-GCC and Keil toolchains use a 32-bit signed time_t. This overflows on 2038-01-19 03:14:07 UTC. Functions mktime() and localtime() will return incorrect results for dates beyond this point.

Mitigations applied in the current codebase:

- NTP input validation caps accepted years at 2037 (year > 2037 is rejected with NAK).
- All localtime() call sites should add a NULL-pointer guard against out-of-range inputs.
- Long-term fix: switch to a toolchain with 64-bit time_t (ARM-GCC 12+ with _USE_TIME_BITS64) or replace stdlib time functions with a custom unsigned-32-bit implementation.

6. USART Protocol

USART1 is configured on remapped pins PB6 (TX) and PB7 (RX) at 115200 baud, 8N1, with receive interrupt enabled. The protocol uses a simple framed format with XOR checksum.

6.1 Frame Format

```
PC → STM32:
  @<payload>#<XOR_byte>

Example (time sync):
  @TIME,2026-03-24 20:38:00#<xor>

XOR_byte = XOR of all bytes in <payload> (excludes @ and #)

STM32 → PC:
  ACK\n   —  frame accepted and processed
  NAK\n   —  checksum failure or invalid content
```

6.2 Receiver State Machine

The ISR implements a three-state machine to parse frames byte-by-byte without blocking the CPU:

IDLE	Receive '@'	Reset buffer and XOR accumulator	RECEIVING
IDLE	Any other byte	Discard silently	IDLE
RECEIVING	Receive '#'	Payload complete; await checksum byte	CHECKSUM
RECEIVING	Buffer overflow (>= 63 bytes)	Discard frame	IDLE
RECEIVING	Any other byte	Append to buffer; XOR into	RECEIVING

		accumulator	
CHECKSUM	Byte matches XOR accumulator	Set rx_complete = 1	IDLE
CHECKSUM	Byte does not match	Set rx_error = 1	IDLE

6.3 NTP Sync Payload

The only defined payload type is TIME. The PC-side script must send the frame immediately upon opening the serial port; the STM32 waits up to 30 seconds before timing out.

```
Payload:  TIME,YYYY-MM-DD HH:MM:SS
Example:  TIME,2026-03-24 20:38:00

Validation rules applied by NTP_SyncTime():
  Year   : 2000 – 2037 (hard cap before int32 overflow)
  Month  : 01 – 12
  Day    : 01 – 31
  Hour   : 00 – 23
  Minute : 00 – 59
  Second : 00 – 59
```

7. OLED Display Layout

The SSD1306 128x64 OLED is driven over software I2C on PB8/PB9. It displays four rows of 8x16 pixel characters (4 rows x 16 columns).

1	CNT:	10-digit RTC counter value	Every tick (1 s)
2	ALM:	Number of configured alarm slots	Static (set at boot)
3	ALRF:	RTC alarm flag (0/1) + '*' heartbeat	Every tick (1 s)
4	MODE:	Current FSM state string	On state change

7.1 Row 4 State Strings

STATE_IDLE	MODE:WAIT ALR
STATE_ALARM (slot N)	MODE:ALM N ON
STATE_STANDBY	MODE:STANDBY
STATE_BOOT_DOWNLOAD	DOWNLOAD MODE / PB11=HIGH / Waiting... / Pull PB11 LOW

STATE_BOOT_NTP	NTP Syncing... (then NTP OK! or NTP FAIL-RTC OK)

8. Power Management

8.1 STANDBY Mode

STM32 STANDBY mode is the deepest available sleep state, with a typical supply current of approximately 2 uA. All SRAM and register state is lost; wake-up causes a full system reset, and main() re-executes from the top.

Wake sources enabled in this firmware:

- RTC alarm: the RTC_ALR register is programmed by ScheduleNextBoundary() before entering standby.
- WKUP pin (PA0): a rising edge on PA0 wakes the device regardless of the RTC alarm state. Enabled via PWR_WakeUpPinCmd(ENABLE).

8.2 Wake and Re-initialisation Sequence

1. main() executes from the top. All peripherals are re-initialised.
2. MyRTC_Init() detects BKP_DR1 = 0xA5A5, skips LSE config, calls RTC_WaitForSynchro().
3. FSM enters STATE_BOOT_CHECK, reads PB11 and PB1, routes to IDLE or NTP.
4. CollectEvents() fires EVT_RTC_ALARM on the first tick because RTC_FLAG_ALR is set from the wake event.
5. FSM_Transition() re-evaluates alarm state and programs the next boundary.

8.3 IDLE_STANDBY_DELAY_SEC

The constant `IDLE_STANDBY_DELAY_SEC` (default: 8) defines how many one-second ticks the system remains in STATE_IDLE before entering STATE_STANDBY. This gives the user time to observe the OLED display after a wake event before the system sleeps again.

9. Boot Modes

Two pins are sampled immediately after peripheral initialisation on every boot (including wake from STANDBY).

PB11	High	Download	FSM enters STATE_BOOT_DOWNLOAD. Infinite loop;

	(float/VCC)	mode	SWD active. Used when connecting a debugger or flashing new firmware.
PB11	Low (GND)	Normal mode	System proceeds normally through NTP check and into the alarm FSM.
PB1	High (float/VCC)	NTP sync	FSM enters STATE_BOOT_NTP. Calls NTP_SyncTime(), waits up to 30 s for a time frame over USART1.
PB1	Low (GND)	Skip NTP	System uses the existing RTC time without attempting synchronisation.

Both pins are configured as pull-up inputs by Key_Init(). A floating (unconnected) pin reads HIGH. Pull the pin to GND to select the LOW behaviour for normal operation.

10. Known Issues and Limitations

KI-01	Medium	Year 2038: 32-bit signed time_t overflows on 2038-01-19. mktime() and localtime() return incorrect results for later dates.	NTP input capped at year 2037. Long-term: use 64-bit time_t or custom time arithmetic.
KI-02	Low	Full power loss (VDD and VBAT simultaneously) resets the backup domain, causing RTC to reinitialise to the hardcoded default date 2023-01-01 23:59:55.	Connect VBAT to a coin cell. Add OLED warning if BKP_DR1 == 0 on boot.
KI-03	Low	Overlapping alarm slots produce undefined behaviour — no overlap detection at runtime.	Detect overlaps at compile time via static_assert (C11) or add runtime check in main().
KI-04	Low	1-second Delay_ms(1000) tick is not compensation for ISR or processing time; clock drift of several seconds per hour is possible.	Use RTC counter delta instead of Delay_ms for tick gating.
KI-05	Info	USART receive buffer is 64 bytes. Payloads longer than 63 bytes are silently discarded.	Increase RX_BUF_SIZE in USART.c if longer payloads are needed.

11. Revision History

1.0	2026-03-20	—	Initial single-alarm implementation with RTC standby cycle.
1.1	2026-03-22	—	Bug fixes: RTC counter read consistency (Bug1-3), alarm flag priority (Bug4).
1.2	2026-03-23	—	USART framed protocol with XOR checksum; NTP sync support added.
2.0	2026-03-24	—	Multi-alarm support; full FSM refactor; this document created.

End of Document